pfi

Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

Course 5 Capstone

WEEK 3 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The scenario: a public file and four questions

A colleague who covers Asian corporates hands you a public file: financial ratios of Taiwanese companies, each row flagged by whether the company went bankrupt.

Which of four models separates them best, on unseen rows?

How sure, with a few dozen bankrupt test rows?

Which ratios does the chosen model lean on?

Where does it fail?

What the answer is not

Not a credit view on any company

Not a rating

Not a probability going forward

Not advice to a lender or an investor

It is a description of a historical, anonymised dataset: what a model shows on it, and where it is wrong.

What you hand in: fifteen deliverables

#	Deliverable
1	ensemblekit complete, pytest passing
2	load_taiwan(path): read, check, clean names
3	The whole file described, hashed checks
4	prepare(frame): drop copies, split 80/20
5	Four models, tuned by randomized search
6	One imbalance decision, with numbers
7	Thresholds for recall 0.70, out of fold
8	The test rows, opened once, with intervals

#	Deliverable
9	A choice before the test rows, nested score
10	SHAP, permutation importance, split counts
11	A results workbook, read back
12	What the model shows, and where it fails
13	Further practice, named
14	One commit per file, then git tag course5
15	An AI-use log

The functions go into one module of your own, taiwan.py, which grows question by question.

The file: UCI dataset 572, Taiwanese bankruptcy

Real, public, anonymised

UCI Machine Learning Repository, dataset 572, CC BY 4.0. The page gives 6,819 instances and 95 features.

"The data were collected from the Taiwan Economic Journal for the years 1999 to 2009. Company bankruptcy was defined based on the business regulations of the Taiwan Stock Exchange."

Citation: the page's own, word for word, in the notebook's data line. DOI: https://doi.org/10.24432/C5004D

What the page does not give

No creator, no paper

No units, no scaling

No horizon for the outcome

No company identifier

A variable list in another order

So the course counts rows, not companies, and uses the file's own names.

What makes it hard: nothing is planted

Names

a leading space on every feature; three with a non-ASCII sign

A constant column

one value in every row

Copies and near copies

columns equal in every row, and others almost

Two scales

columns that hold small ratios and huge numbers

A row order

the file is not in random order

A rare outcome

and a small test set

Section 3 describes the whole file before any split. Nothing computed there builds a feature or chooses anything.

Twelve sections, three sittings

First sitting

1 Your module

2 Load and check

3 Describe the file

4 Prepare

Check, describe, split

Second sitting

5 Four models

6 Imbalance

7 Thresholds

8 The choice

Model and choose

Third sitting

9 The test rows

10 SHAP and importance

11 The workbook

12 Description and log

Test once, describe

The searches try 22 settings, five folds each, and the nested score repeats a search in five outer folds: minutes per cell on one thread.

Three roles of rows

Training rows

fit

every model, transform, and resampler

Folds of the training rows

choose

settings, treatment, thresholds, the model

Test rows

report

once, in section 9, after the choice

```
train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
```

Even a list of columns can leak

Section 4 makes one list from the training rows only, and asks what would change if it came from the whole file.

Four models, each a function

build_logistic()

- QuantileTransformer(normal)
- LogisticRegression
- newton-cholesky, tol=1e-8

build_forest()

- RandomForestClassifier
- n_jobs=1
- random_state=SEED

build_xgboost()

- XGBClassifier
- n_jobs=1
- random_state=SEED

build_lightgbm()

- LGBMClassifier
- n_jobs=1, verbose=-1
- random_state=SEED

```
RandomizedSearchCV(..., scoring="average_precision", cv=mlkit.cv_folds(), random_state=SEED, n_jobs=1)
```

Each returns an unfitted pipeline whose last step is named model. The distributions are a complete cell: do not change them.

Three kinds of number, kept apart

The search's best score

The best of several noisy numbers: it flatters. Compare four models with it, beside the fold standard deviation.

The nested score

The whole search, repeated inside each outer fold. The honest score of the chosen model.

Test rows, with intervals

A bootstrap interval on each test number: the noise from which rows were scored, not from refitting.

If a gap between two models is smaller than the noise, say so: that is a finding, not a failure.

The rare outcome, and the threshold

One decision, three treatments

No treatment

Class weights

SMOTE inside ImbPipeline

Compared on the folds: average precision, precision at the desk's recall, and the mean output. A weighted or resampled output is the model's score.

Threshold: recall of at least 0.70

```
threshold, oof = ensemblekit.oof_threshold(
    model, X_train, y_train)
```

Out of fold, on the training rows; never the test rows. An illustration of the trade between catching and over-flagging, not a lending rule.

How the checks work

A check holds a short code, never the answer

```
check('Question 3.1', answer_3_1, 'a short code')
```

```
correct
```

```
not yet. You have ...
```

```
not attempted yet
```

Store the full number; check rounds to the places asked. Text without capitals. Several values in a tuple.

Decisions change the numbers

Checks after section 4 assume the course's decisions. "Stuck after trying?" blocks hold a decision, never a count: make your own first.

43 questions and 38 check lines, in the order an analyst works.

What an output and a SHAP value may be called

Allowed names

model probability of bankruptcy in this file

flagged at threshold t

the model's score (weighted or resampled)

Never called

a credit view on any company

a rating

an investment decision

Allowed names

SHAP value: contribution to this row's log-odds

contribution to the model's score

mean absolute SHAP value

Never called

reason for default or reason code

cause or driver

effect of changing the feature

SHAP values add up in log-odds, not probability: assert_additive before reading any of them.

Every comparison: a chart and a table

Provided chart cells

5.6 to 7.1	Searches, treatments, thresholds: styled table and bars
9.1, 9.2	Test rows: styled table, bars, intervals beside the folds
9.3	Heatmaps on three sets of rows; ROC and PR curves, all four
10.5	SHAP, permutation, split counts as bars
12	Every model on every set of rows: table and bars

Each prints not attempted yet until the answers it draws from are stored.

A First Look, before section 1

One ratio on its own scale

A column with two scales

Bankrupt rows in the file's order

One ratio, bankrupt against the rest

Read from the file as it arrived. They answer no question.

No chart from the work is shown in this video: the charts are yours to draw.

The workbook, the description, Git

The workbook

Four sheets, read back. Excel rebuilds the confusion matrix, the nested mean and sd, the interval, and one SHAP row.

The description

300 to 500 words, checked by assert_described: no forecast words, no cause words.

Git

One commit per deliverable that writes a file, then git tag course5.

Further practice: Freddie Mac's loan-level data. Free with registration; redistribution is prohibited, so download it yourself.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows choose, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: a split that returns parts of the right sizes. Section 3 already told you what is wrong with it.

Code that runs is not the same as code that is right.

What you can do now

- Say why an average of trees beats one tree, and when not
- Tune without fooling yourself; read the noise between folds
- Treat a rare outcome, and say what it does to an output
- Open the test rows once, with an interval
- Describe a model with SHAP values, never as causes

Next keep your my_bondmath folder up to date, and bring it with you.